Benchmarking Spatio-Temporal Graph Neural Networks for Time Series Forecasting Across Heterogeneous Graph Domains

Gabriel F. Costa[1 [0009−0001−2714−3365]], Eduardo J. S. Luz[1 [0000−0001−5249−1559]], and Vander L. S. Freitas[1 [0000−0001−7989−0816]]

[1] Federal University of Ouro Preto
CEP 35400-000 – Ouro Preto – MG – Brazil
PPGCC – Postgraduate Program in Computer Science
vander.freitas@ufop.edu.br

Abstract. In this work, we systematically evaluate 21 spatio-temporal Graph Neural Network (STGNN) architectures under a unified pipeline across four heterogeneous datasets, comparing architectural families defined by their temporal modules (convolutional, recurrent, and attention-based) to assess their impact on node-level time series forecasting performance. The experiments evidence that different temporal mechanisms yield distinct predictive behaviors, with attention-based models showing advantages primarily in long-range or non-local dependency regimes, and that graph spectral properties are associated with relative performance. All experiments use consistent splits and various seeds, and graph spectra are analyzed via the Laplacian. Results show statistical differences across models but no universal winner; convolutional models remain competitive, while attention-based models tend to perform better in long-dependency settings, indicating that effectiveness is dataset-dependent and associated with spectral characteristics. Additionally, the results suggest a descriptive alignment between spectral profiles and predictive performance.

Keywords: Spatio-Temporal Graph · Graph Neural Networks · Time Series Forecasting · Graph Spectral Analysis

1 Introduction

The increasing availability of time-series data in domains such as epidemiology, urban mobility, and large-scale information systems has intensified the need for models that capture complex spatio-temporal dependencies [1, 2]. This work addresses a central gap: there is no unified understanding of how temporal modeling within Graph Neural Networks (GNNs) is associated with predictive performance, particularly across different graph properties. In real-world systems, observations evolve over interconnected entities rather than independently, which are naturally represented as graphs, where nodes denote entities and edges encode relationships [3]. GNNs extend this representation by learning from both

node features and graph structure [4, 5]. Leveraging this, spatio-temporal GNNs (STGNNs) jointly model spatial dependencies and temporal dynamics; yet, how architectural choices in temporal modules affect efficacy remains underexplored and is the focus of our inquiry.

STGNNs combine a graph-based spatial operator with a temporal modeling mechanism [5]. The spatial part uses message passing, where nodes aggregate information from neighbors, often via graph convolutions or diffusion operators. These can be seen as graph filters that emphasize low-frequency components, smoothing signals over the graph. The temporal part captures how node representations change over time, using recurrent neural networks, convolutional neural networks, or attention mechanisms. While recurrent and convolutional models rely on local propagation and filtering, attention-based models allow adaptive, often non-local interactions by dynamically weighting relevant nodes and time steps.

Different architectural families implement this coupling in distinct ways. Diffusion Convolutional Recurrent Neural Networks (DCRNN) [6] integrate diffusion-based graph convolutions within recurrent units, embedding spatial propagation into temporal updates. Convolutional approaches, such as STGCN [7] and Graph WaveNet [8], decouple space and time by combining graph and temporal convolutions, improving parallelism and enlarging the temporal receptive field. Attention-based models, such as GMAN [9], replace fixed aggregation with adaptive attention, capturing long-range dependencies and dynamically focusing on relevant nodes and time steps, particularly in non-stationary settings. Despite these advances, how different temporal mechanisms affect performance under controlled conditions remains unclear.

Another question concerns how graph structure interacts with temporal modeling. Graphs vary in connectivity, sparsity, and topology, affecting information flow via message passing. Spectrally, graph convolutional operators act as low-pass filters, favoring smooth signals consistent with the graph [10, 4]. In these cases, local message passing can capture most of the information. However, when graph signals have more mid or high-frequency content, weaker locality, or non-local interactions, local propagation may fall short, making flexible mechanisms like attention more beneficial.

In this work, we investigate how the choice of temporal modeling mechanism in STGNNs—whether convolutional, recurrent, or attention-based—relates to the spectral and structural properties of the underlying graph signals. We systematically evaluate multiple STGNN architectures using a unified, controlled pipeline across heterogeneous real-world datasets, standardizing data splits, training procedures, and hyperparameters to reduce confounding factors in the comparison of architectural design choices. Additionally, we incorporate spectral analysis based on the graph Laplacian to characterize signal smoothness and frequency content, examining these properties through an interpretative hypothesis that the alignment between temporal mechanisms and graph structure may be associated with predictive effectiveness.

The aim of this work is to provide an interpretative link between model design and dataset characteristics. Our results indicate that distinct temporal mechanisms give rise to distinct predictive behaviors, and that attention-based models may be advantageous in scenarios with long-range, non-local, or non-stationary dependencies beyond local propagation. Moreover, graph spectral properties, specifically the balance between low-frequency dominance and mid/high-frequency components, appear to be associated with variations in STGNN performance, considering the datasets analyzed.

2 Materials and Methods

2.1 Graph Analysis

A graph can be defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{v_1, \ldots, v_N\}$ and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ denote the sets of nodes and edges, with $N = |\mathcal{V}|$ [3]. In the attributed and temporal setting, this definition extends to $\mathcal{G}(t) = (\mathcal{V}(t), \mathcal{E}(t), \mathbf{X}(t))$, where $\mathbf{X}(t) \in \mathbb{R}^{N(t) \times F}$ is the node feature matrix, associating each node $v_i \in \mathcal{V}(t)$ with a feature vector $\mathbf{x}_i(t) \in \mathbb{R}^F$ [5], i.e. a graph signal over the nodes.

The graph structure is represented by an adjacency matrix $\mathcal{A}(t) \in \mathbb{R}^{N(t) \times N(t)}$, whose entries encode edge presence or weights between nodes $(v_i, v_j) \in \mathcal{E}(t)$, with $\mathcal{A}_{ij}(t) > 0$ if $(v_i, v_j) \in \mathcal{E}$ and $\mathcal{A}_{ij}(t) = 0$ otherwise [11]. In unweighted graphs, $\mathcal{A}_{ij}(t) \in \{0, 1\}$, while in weighted graphs $\mathcal{A}_{ij}(t) = w_{ij}(t)$.

In static graphs, $\mathcal{V}(t) = \mathcal{V}$ and $\mathcal{E}(t) = \mathcal{E}$ remain fixed, whereas in dynamic graphs the topology evolves over time, allowing nodes and edges to appear, disappear, or change. In spatio-temporal settings, both the graph structure and node attributes may change over time, resulting in a sequence of attributed graphs:

$$\{\mathcal{G}(t)\}_{t=1}^{T}. \tag{1}$$

The degree k_i of a node v_i is the number of incident edges, and $\langle k \rangle$ is the average degree [3, 11], given by $k_i = \sum_{j=1}^{N} \mathcal{A}_{ij}$ and $\langle k \rangle = \frac{2|\mathcal{E}|}{N}$, where $|\mathcal{E}|$ is the number of edges. The graph diameter $\rho_{\max}$ is the maximum shortest-path distance between any pair of nodes, where ρ_{ij} is the distance between nodes v_i and v_j, i.e., $\rho_{\max} = \max_{i,j} \rho_{ij}$.

Beyond these structural measures, global graph properties can be analyzed through spectral graph theory [10, 12]. The graph Laplacian is defined as $\mathcal{L} = \mathcal{D} - \mathcal{A}$, where $\mathcal{D}$ is the degree matrix with entries $\mathcal{D}_{ii} = k_i$. For undirected graphs, $\mathcal{L}$ is symmetric and positive semi-definite, ensuring real eigenvalues and an orthonormal eigenbasis. For directed graphs, the adjacency matrix is generally asymmetric. A common solution is to symmetrize it, e.g., $\tilde{\mathcal{A}} = \frac{\mathcal{A}+\mathcal{A}^\top}{2}$, and define the normalized Laplacian $\mathcal{L} = \mathbf{I} - \mathbf{D}^{-1/2}\tilde{\mathcal{A}}\mathbf{D}^{-1/2}$.

In dynamic settings, where the graph evolves over time, the Laplacian becomes time-dependent, $\mathcal{L}(t)$. Here, we use a time-aggregated adjacency matrix to define a representative Laplacian that captures average structural properties while preserving interpretability.

The graph Laplacian eigenvalues define frequencies: small μ_k correspond to low-frequency smooth components, while large μ_k capture high-frequency variations. The smoothness of a graph signal $\mathbf{x} \in \mathbb{R}^N$ is quantified by the Dirichlet energy $\mathscr{E}_D = \sum_{k=1}^{N} \mu_k \hat{x}_k^2$, where $\hat{x}_k = \mathbf{u}_k^\top \mathbf{x}$ are the Graph Fourier coefficients [10, 12, 13]. The spectral energy is $\mathscr{E}_k = \hat{x}_k^2$, with normalized form $\tilde{\mathscr{E}}_k = \frac{\mathscr{E}_k}{\sum_{j=1}^{N} \mathscr{E}_j}$.

To summarize the spectral distribution, we partition the spectrum $\Lambda = \{\mu_k\}_{k=1}^{N}$ into low, mid, and high-frequency bands:

$$\mathscr{E}_L = \sum_{k=1}^{\lfloor N/3 \rfloor} \tilde{\mathscr{E}}_k, \quad \mathscr{E}_M = \sum_{k=\lfloor N/3 \rfloor + 1}^{\lfloor 2N/3 \rfloor} \tilde{\mathscr{E}}_k, \quad \mathscr{E}_H = \sum_{k=\lfloor 2N/3 \rfloor + 1}^{N} \tilde{\mathscr{E}}_k, \tag{2}$$

which quantify how energy is distributed across smooth, intermediate, and rapidly varying components.

2.2 Graph Neural Networks for Time Series Forecasting

Time series forecasting predicts future values from past observations. Given graph signals $\mathbf{x}(t)$ for $t = 1, \ldots, T$, the goal is to estimate the next H values $\mathbf{x}(T+1), \ldots, \mathbf{x}(T+H)$ over a prediction horizon H [1, 14]. A standard formulation uses a fixed temporal context (lags), denoted as F, representing the number of past observations per node at each time step. Training samples are generated via a sliding window, converting the sequence into supervised data while preserving temporal order. Forecasting thus consists of predicting future node-level signals from historical graph observations, capturing both temporal dynamics and spatial dependencies.

Graph Neural Networks (GNNs) extend deep learning to non-Euclidean domains by allowing each node to aggregate information from its neighbors [5]. Unlike shallow embedding methods, which assign fixed lookup vectors to nodes, modern GNNs learn deep representations through stacked message-passing layers. At each layer, node states are updated by combining their current representations with permutation-invariant aggregations of neighboring information, such as sum, mean, max, or attention-based weighting. This aggregation–update process produces task-dependent embeddings that jointly encode node attributes and graph connectivity.

The forecasting task consists of predicting future graph signals from historical observations. A spatio-temporal GNN f_θ, parameterized by θ, learns the mapping:

$$f_\theta : (\mathcal{G}, \mathbf{X}_{t-L+1:t}) \mapsto \hat{\mathbf{X}}_{t+1:t+H}, \tag{3}$$

where $\mathbf{X}_{t-L+1:t} = \{\mathbf{x}(t-L+1), \ldots, \mathbf{x}(t)\}$ denotes the sequence of observed graph signals over a temporal context of length L and H is the forecasting horizon. The output $\hat{\mathbf{X}}_{t+1:t+H}$ represents the predicted future signals. STGNN architectures typically combine two components: a spatial operator and a temporal operator. The spatial operator g_ϕ performs message passing over the graph:

$$\mathbf{U}^{(t,\ell)} = g_\phi \left(\mathbf{U}^{(t,\ell-1)}, \mathcal{A} \right), \tag{4}$$

where $\mathbf{U}^{(t,\ell)} \in \mathbb{R}^{N \times d}$ denotes the node embedding matrix at time t and spatial layer ℓ, d is the latent (hidden) dimension, and ϕ represents the learnable parameters of the spatial operator. The initial representation is typically $\mathbf{U}^{(t,0)} = \mathbf{x}(t)$ or its projected version.

The temporal operator h_ψ models the evolution of node representations across time:

$$\mathbf{Z}^{(t)} = h_\psi \left(\mathbf{U}^{(\tau,K_s)}{}_{\tau \leq t} \right), \tag{5}$$

where ψ denotes the learnable parameters of the temporal operator, K_s is the number of spatial layers, and τ denotes a temporal index such that $\mathbf{U}^{(\tau,K_s)}{}_{\tau \leq t}$ represents the sequence of node embeddings up to time t. The output $\mathbf{Z}^{(t)} \in \mathbb{R}^{N \times d}$ represents the temporally aggregated node representation. Predictions are obtained through a regression head p_ω:

$$\hat{\mathbf{x}}(t) = p_\omega \left(\mathbf{Z}^{(t)} \right), \tag{6}$$

where ω denotes the learnable parameters of the output layer.

Convolutional modules capture spatial and temporal dependencies using localized filters [5]. Representative architectures include STGCN [7], Graph WaveNet [8], MTGNN [15], LSGCN [16], SLCNN [17], and AAGCN [18]. They combine spatial graph convolutions with temporal convolutional mechanisms such as gated temporal convolutions, dilated causal convolutions, or inception-style filters. Some approaches (e.g., Graph WaveNet and MTGNN) incorporate adaptive or learned adjacency matrices, while others rely on spectral formulations such as Chebyshev polynomials.

Recurrent modules propagate information through time via hidden states. Representative architectures include DCRNN [6], TGCN [19], GConvGRU and GConvLSTM [20], GCLSTM [21], MPNN-LSTM [22], DyGrAE [23], and EvolveGCN [24]. They integrate recurrent units such as GRU or LSTM with graph convolutions or message passing schemes. In particular, DCRNN employs diffusion convolution within a GRU framework, while EvolveGCN dynamically updates GNN parameters over time.

Attention-based models assign dynamic importance to nodes and time steps. Representative architectures include GMAN [9], TGAT [25], STAEformer [26], STGNN [27], STGraformer [28], and CaST [29]. These models rely on self-attention or transformer mechanisms to capture long-range dependencies, often incorporating temporal encodings, graph-aware biases, or sparsification strategies. For instance, TGAT uses continuous-time encoding, STAEformer applies adaptive sparse attention, and CaST leverages frequency-domain representations. In spatio-temporal settings, attention operates across nodes, time steps, or both, enabling flexible modeling of non-local dependencies and complex temporal patterns.

2.3 Datasets

We evaluate the proposed framework on four real-world datasets from PyTorch Geometric Temporal [30]: Chickenpox Hungary, WikiMaths, England COVID-

19, and Montevideo Bus. These public and reproducible datasets span different domains, temporal resolutions, graph sizes, and structural regimes, comprising three static graphs and one dynamic graph, with directed/undirected and weighted/unweighted relations. Node features are built from lagged node-level observations, and targets correspond to future values over the forecasting horizon H. Although not exhaustive, this set provides a compact benchmark for controlled comparison across heterogeneous graph domains. Table 1 summarizes their main structural, temporal, and spectral properties.

Table 1. Graph statistics and spectral properties. $\mathcal{G}_{\text{type}}$: graph type (S = static, D = dynamic); D: directed (Y = yes, N = no); w: weighted (Y = yes, N = no); Δt: temporal resolution (W = weekly, D = daily, H = hourly); T: number of time steps; $|\mathcal{V}|$: number of nodes; $|\mathcal{E}|$: number of edges (average for dynamic graphs); $\langle k \rangle$: average degree; $\rho_{\max}$: graph diameter; $\mathscr{E}_L, \mathscr{E}_M, \mathscr{E}_H$: spectral energy in low, mid, and high-frequency bands; $\mathcal{E}_D \pm \sigma$: mean and standard deviation of Dirichlet energy.

Dataset	$\mathcal{G}_{\text{type}}$	D	w	Δt	T	$\lvert\mathcal{V}\rvert$	$\lvert\mathcal{E}\rvert$	$\langle k \rangle$	$\rho_{\max}$	$\mathscr{E}_L$	$\mathscr{E}_M$	$\mathscr{E}_H$	$\mathcal{E}_D \pm \sigma$
Chickenpox	S	N	N	W	517	20	41	4.10	6.00	0.4387	0.2785	0.2828	16.26 ± 17.35
WikiMaths	S	Y	Y	D	723	1,068	27,079	50.71	4.00	0.6562	0.1702	0.1736	629.89 ± 348.82
England	D	Y	Y	D	53	129	663.89	10.29	5.79	0.6370	0.1856	0.1774	70.33 ± 24.16
Montevideo	S	Y	Y	H	740	675	690	2.04	155.00	0.4267	0.2902	0.2831	582.78 ± 534.08

The Chickenpox Hungary dataset contains weekly chickenpox case counts across 20 Hungarian counties from 2004 to 2014. The graph is static and undirected. Nodes represent counties, and edges encode neighborhood relations. The forecasting task predicts future case counts for each county. This is a small epidemiological graph with local spatial dependencies, regular sampling, and seasonal dynamics. Its diameter $\rho_{\max} = 6.00$ indicates relatively short paths between counties. Its spectral distribution suggests both global trends and localized variation.

The WikiMaths dataset contains daily visits to vital mathematics-related Wikipedia pages from March 2019 to March 2021. Nodes represent Wikipedia pages. Directed weighted edges represent hyperlinks, weighted by the number of links from source to target pages. The forecasting task predicts future daily visits for each page. This is the largest and densest graph in the benchmark, with 1,068 nodes and 27,079 edges. Its small diameter $\rho_{\max} = 4.00$ reflects high global connectivity. Its strong low-frequency concentration indicates coherent variation over a highly connected knowledge graph.

The England COVID-19 dataset models daily case dynamics across NUTS3 regions in England. The graph is dynamic, directed, and weighted, with fixed nodes representing regions and time-varying edges encoding daily mobility flows from Facebook Data for Good disease-prevention maps. The forecasting task predicts future daily case counts for each region. Its time-averaged graph diameter is $\rho_{\max} = 5.79$, indicating moderate reachability despite time-varying mobil-

ity. The dataset includes non-stationary epidemic behavior and abrupt temporal changes.

The Montevideo Bus dataset contains hourly passenger inflow at bus stops from 11 bus lines in Montevideo during October 2020. Nodes represent bus stops. Edges indicate route connectivity, and edge weights represent road distance. The forecasting task predicts future passenger inflow for each stop. The graph is sparse and elongated, with a large diameter $\rho_{\max} = 155.00$. This reflects long-range propagation over transportation routes. Its hourly signal captures fine-grained urban mobility dynamics. Its more balanced spectral distribution suggests both global traffic patterns and localized stop-level variation.

2.4 Experimental Setup

We adopt a mini-batch training strategy. Each batch contains spatio-temporal samples of shape (B, L, N, F). Here, $B \in \mathbb{N}$ denotes the batch size, $L \in \mathbb{N}$ is the temporal context length, $N \in \mathbb{N}$ is the number of nodes, and $F \in \mathbb{N}$ is the number of features per node. In all datasets used in this benchmark, the number of nodes remains fixed across temporal snapshots; therefore, graph dynamics, when present, correspond to time-varying edges or edge weights, but not to the appearance or disappearance of vertices. Samples are generated using a sliding window over time. For a given time index t, the input and target are defined as:

$$\mathbf{X}_{t-L+1:t} \to \mathbf{X}_{t+1:t+H}, \tag{7}$$

where $\mathbf{X} \in \mathbb{R}^{L \times N \times F}$ contains the past L observations, and $\mathbf{X}_{t+1:t+H} \in \mathbb{R}^{H \times N \times F}$ denotes the target tensor over a forecasting horizon $H \in \mathbb{N}$.

Running Example of a Model Input. To exemplify the model input, let F denote the feature (lag) size and L the temporal context. At time t, a node (e.g., node 2) is represented by a feature vector of size $F = 5$, containing the most recent observations:

$$\mathbf{x}_2(t) = \{x_{t-4}, x_{t-3}, x_{t-2}, x_{t-1}, x_t\}. \tag{8}$$

To predict the future, we consider a context of $L = 5$ past time steps, i.e., $\{\mathbf{x}_2(t), \mathbf{x}_2(t-1), \ldots, \mathbf{x}_2(t-4)\}$, where each vector has the same structure:

$$\mathbf{x}_2(t-1) = \{x_{t-5}, \ldots, x_{t-1}\}, \quad \mathbf{x}_2(t-2) = \{x_{t-6}, \ldots, x_{t-2}\}, \text{ etc.} \tag{9}$$

Although each vector $\mathbf{x}_2(t)$ contains an implicit temporal ordering, we employ here the LSTM, GRU, RNN, or Transformers modules to model dependencies *across* these vectors (i.e., across time steps), rather than within them.

Each input vector of size F is first projected into a lower-dimensional space of size d via a linear transformation. Spatial dependencies are then captured through graph convolutions over neighboring nodes, while temporal dependencies are learned over the sequence of latent representations across the L time steps. In our experiments, we set $F = L$.

Datasets are split chronologically: training (70%), validation (15%), and test (15%) sets. This prevents information leakage. All time series are standardized using z-score normalization fitted on the training set:

$$\tilde{x}_i^{(t)} = \frac{x_i^{(t)} - \mu_i}{\sigma_i}, \tag{10}$$

where μ_i and σ_i denote the empirical mean and standard deviation of node i.

Training Configuration. Models are trained using the AdamW optimizer [31] for up to 200 epochs with batch size $B = 32$. Early stopping is applied with patience of 20 epochs. The learning rate follows a linear warmup (5 epochs) followed by cosine annealing [32]. Gradient clipping (norm = 1.0) and mixed precision training are employed. Each configuration is evaluated over $S = 10$ independent seeds. The full implementation used in this work is publicly available[1].

Temporal Context and Forecasting Horizon. We evaluate temporal contexts $L \in \{2, 4, 8, 12\}$ and forecasting horizons $H \in \{1, 5, 10\}$. Larger values of L increase the temporal receptive field, while larger values of H increase prediction difficulty. For extended settings, we use $L = 50$ and $H = 20$ on the larger datasets, while for the smaller *England COVID* dataset we use $L = 20$ and $H = 10$ to avoid issues in the rolling window scheme.

Unified Spatio-Temporal Pipeline. The input tensor is $\mathbf{X} \in \mathbb{R}^{B\times L\times N\times F}$. At each time t, each node is represented by a vector in $\mathbb{R}^F$ encoding recent observations, and the model processes a sequence of L such vectors, capturing dependencies across time steps (rather than within each vector). An input projection maps features into a latent space:

$$\mathbf{X}' = \mathbf{X}\mathcal{W}_{in}, \quad \mathcal{W}_{in} \in \mathbb{R}^{F\times d}, \tag{11}$$

where d denotes the hidden dimension and $\mathcal{W}_{in}$ is a learnable linear transformation applied independently at each time step. We compare the impact of the input projection by evaluating each architecture with and without this transformation, i.e., using either $\mathbf{X}' = \mathbf{X}\mathcal{W}_{in}$ or the original input $\mathbf{X}$ directly, keeping all other components and hyperparameters unchanged. A spatio-temporal block models dependencies through:

$$\mathbf{U}^{(t,\ell)} = g_\phi\left(\mathbf{U}^{(t,\ell-1)}, \mathcal{A}(t)\right), \quad \mathbf{Z}^{(t)} = h_\psi\left(\mathbf{U}^{(\tau,K_s)}{}_{\tau=t-L+1}^{t}\right), \tag{12}$$

where $\mathbf{U}^{(t,\ell)} \in \mathbb{R}^{B\times N\times d}$ are node embeddings at time t and spatial layer ℓ, K_s is the number of spatial layers, and $\tau \in \{t-L+1, \ldots, t\}$. The representation $\mathbf{Z}^{(t)} \in \mathbb{R}^{B\times N\times d}$ corresponds to temporally aggregated node embeddings, encoding joint spatio-temporal information.

[1] Repository: https://github.com/gabrielxcosta/Spatiotemporal-GNN-Forecasting

For static datasets, $\mathcal{A}(t) = \mathcal{A}$ for all time steps. For the dynamic dataset, the node set remains fixed, so $\mathcal{A}(t) \in \mathbb{R}^{N \times N}$, although its entries may vary due to changes in edges or edge weights. Missing edge weights are set to one. Architectures typically include residual connections and normalization layers to stabilize training.

The last time step gives $\mathbf{h} = \text{Dropout}(\text{ReLU}(\mathbf{Z}^{(t)}))$, the final representation, and predictions are computed as:

$$\hat{\mathbf{X}} = \mathbf{h}\mathcal{W}_{out}, \quad \mathcal{W}_{out} \in \mathbb{R}^{d \times H}, \quad \hat{\mathbf{X}} \in \mathbb{R}^{B \times N \times H}, \tag{13}$$

where $\mathcal{W}_{out}$ is a learnable linear mapping from the latent space to the forecasting horizon.

Evaluation Metrics and Statistical Significance Analysis. Performance is evaluated using MSE, RMSE, MAE, and R^2. Statistical significance is assessed with the Friedman test [33] followed, when applicable, by the Nemenyi post-hoc test [34], and summarized using Critical Difference (CD) diagrams. For the input projection ablation, we report $\Delta\text{RMSE} = \text{RMSE}_{\text{noIP}} - \text{RMSE}_{\text{IP}}$, where positive values indicate worse performance without projection. Spectral trends are examined descriptively across datasets, regimes, architectural families, normalized RMSE values, and ranks.

2.5 Hyperparameter Configuration

To ensure fair comparison, only architecture-dependent hyperparameters (*Hidden size*) are varied: $d \in \{32, 64\}$. Other hyperparameters follow standard defaults: attention models use 4 heads, spectral methods use polynomial order $K = 2$. To control computational costs, which sometimes could exceed 24GB of VRAM, we use a neighborhood size fixed at 20, which corresponds to the maximum number of neighbors considered per node during message passing, limiting the local connectivity. Additionally, the Adaptive mechanisms use Top-$K = 32$ and truncation 16 where applicable.

2.6 Architectures

We evaluate a diverse set of spatio-temporal graph neural networks (STGNNs) grouped by their temporal modeling mechanisms. Table 2 summarizes the architectures, indicating their temporal operator h_ψ and spatial operator g_ϕ.

3 Results

The statistical results for Short-Mid and Long-Range predictions are shown in Figures 1 and 2, respectively, while Table 3 reports absolute RMSE values. In the Short–Mid regime, most architectures perform similarly, with overlapping standard deviations and large indistinguishable groups, as shown in Figure 1. Still,

Table 2. STGNN architectures grouped by temporal operator h_ψ and spatial operator g_ϕ. The column *Family* indicates the type of temporal modeling: A (Attention-based), C (Convolutional), and R (Recurrent).

Model	Year	Family	h_ψ	g_ϕ
CaST [29]	2023	A	Self-Attention + FFT	Message Passing
GMAN [9]	2020	A	Multi-head Attention	Spatial Attention
STAEformer [26]	2023	A	Self-Attention	Adaptive Attention
STGNN [27]	2020	A	Temporal Attention	GNN
STGraformer [28]	2021	A	Transformer	Graph Attention
TGAT [25]	2022	A	Time Encoding + Attention	Neighborhood Attention
AAGCN [18]	2024	C	Temporal Conv	Adaptive GCN
GraphWaveNet [8]	2019	C	Dilated Conv	Graph Conv
LSGCN [16]	2020	C	GLU + Conv	Chebyshev GCN
MTGNN [15]	2020	C	Dilated/Inception Conv	Graph Learning
SLCNN [17]	2020	C	Temporal Conv	Chebyshev GCN
STGCN [7]	2018	C	Temporal Conv	Chebyshev GCN
DCRNN [6]	2018	R	GRU	Diffusion Conv
DyGrAE [23]	2018	R	LSTM	GCN
EvolveGCN-H [24]	2020	R	GRU	GCN
EvolveGCN-O [24]	2020	R	GRU	GCN
GCLSTM [21]	2018	R	LSTM	Chebyshev GCN
GConvGRU [20]	2019	R	GRU	Chebyshev GCN
GConvLSTM [20]	2019	R	LSTM	Chebyshev GCN
MPNN-LSTM [22]	2020	R	LSTM	Message Passing
TGCN [19]	2019	R	GRU	GCN

LSGCN, GMAN, STGNN, and STGraformer often achieve the lowest RMSE values, especially on Chickenpox and WikiMaths.

Regarding computational cost, the average training time per epoch (c_{ep}) varies considerably across architectures. Attention-based models such as TGAT and STAEformer are among the fastest, whereas recurrent architectures based on graph recurrent units and some convolutional models require substantially longer training times. This indicates that predictive performance and computational efficiency are not necessarily aligned, highlighting a trade-off between accuracy and training cost.

To examine the relation between graph spectra and architectural behavior, we use a descriptive spectral–architectural alignment analysis based on normalized RMSE, family rankings, and performance dispersion across datasets and regimes. Datasets are grouped by spectral profile, and the observed separation among architectural families is compared with the expected behavior from these profiles. Due to space constraints, the complete calculations are provided in the GitHub repository. The results show no clear alignment in the Short-Mid regime and only partial alignment in the Long-Range regime.

From a spectral perspective, WikiMaths and EnglandCOVID are dominated by low-frequency components, while Montevideo Bus exhibits a more balanced spectral distribution. Correspondingly, the former datasets show smaller performance differences among architectures, whereas the latter presents greater

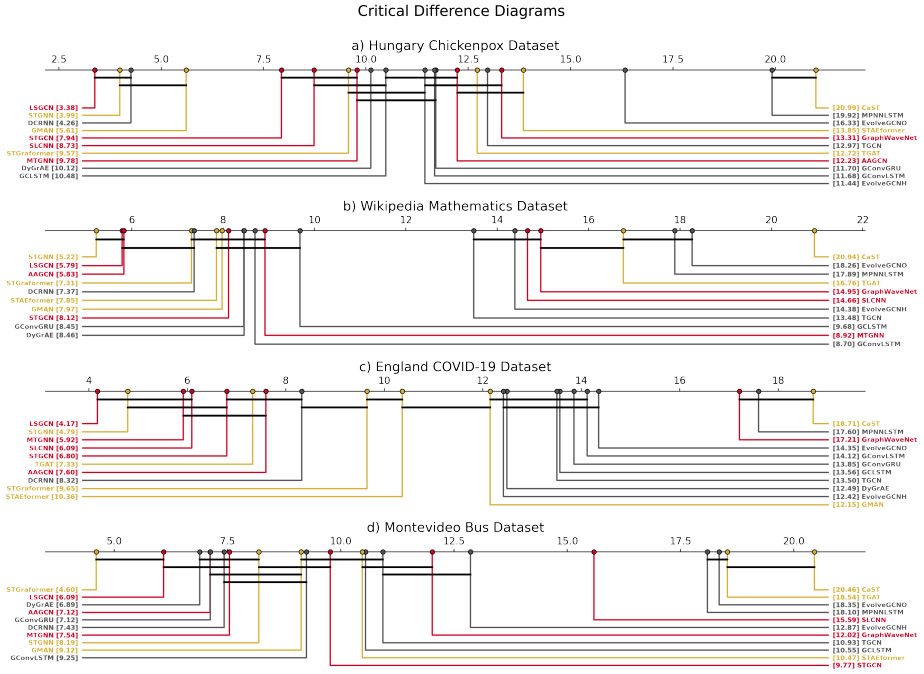


Fig. 1. Critical Difference (CD) diagrams for Short-Mid forecasting regimes. Architectures are ranked according to average RMSE (lower is better), where the axis represents relative ranking positions rather than absolute error values. Models connected by horizontal black bars are not statistically different at $\alpha = 0.05$. Colors indicate architectural families: convolutional (red), recurrent (gray), and attention-based (yellow).

Table 3. Test RMSE (± std across seeds) and average training time per epoch (c_{ep}, seconds) in the Short-Mid and Long-Range regimes. c_{ep} is computed as the mean runtime per epoch across runs for each architecture and regime. Bold values indicate the lowest RMSE per dataset and the lowest c_{ep} per regime.

Architecture	Short-Mid					Long-Range				
	Chickenpox	WikiMaths	EnglandCOVID	MontevideoBus	c_{ep}	Chickenpox	WikiMaths	EnglandCOVID	MontevideoBus	c_{ep}
AAGCN	0.3253 ± 0.2934	0.2958 ± 0.2707	0.4650 ± 0.3212	0.3569 ± 0.3504	1.48	0.2667 ± 0.0639	0.2999 ± 0.0884	0.3211 ± 0.0850	0.2014 ± 0.0531	7.00
CaST	0.6140 ± 0.1122	0.5646 ± 0.1619	0.7303 ± 0.2085	0.4844 ± 0.2735	3.21	0.6886 ± 0.0562	0.7650 ± 0.0265	1.0110 ± 0.0488	0.3970 ± 0.0376	6.40
DCRNN	0.3080 ± 0.2941	0.3017 ± 0.2974	0.4753 ± 0.3177	0.3509 ± 0.3619	2.88	0.2515 ± 0.0617	0.2620 ± 0.0782	0.3294 ± 0.2785	0.1955 ± 0.0552	8.48
DyGrAE	0.3251 ± 0.2842	0.3063 ± 0.2736	0.5658 ± 0.2958	0.3555 ± 0.3565	1.08	0.2980 ± 0.0623	0.2981 ± 0.0737	0.5128 ± 0.1129	0.2033 ± 0.0528	4.87
EvolveGCNH	0.3290 ± 0.2810	0.3262 ± 0.2806	0.5935 ± 0.2547	0.3698 ± 0.3476	2.95	0.3037 ± 0.0602	0.3104 ± 0.0682	0.5930 ± 0.0941	0.2223 ± 0.0535	8.62
EvolveGCNO	0.3686 ± 0.2575	0.3866 ± 0.2741	0.6160 ± 0.2819	0.4062 ± 0.3304	1.06	0.3824 ± 0.0573	0.4710 ± 0.0702	0.6409 ± 0.0798	0.2948 ± 0.0663	3.55
GCLSTM	0.3269 ± 0.2828	0.3096 ± 0.2842	0.5818 ± 0.2952	0.3595 ± 0.3556	2.34	0.3036 ± 0.0672	0.2958 ± 0.0743	0.5489 ± 0.0939	0.2117 ± 0.0560	9.48
GConvGRU	0.3283 ± 0.2834	0.3057 ± 0.2775	0.5806 ± 0.2940	0.3565 ± 0.3554	4.15	0.2909 ± 0.0610	0.3024 ± 0.0711	0.5369 ± 0.0934	0.2059 ± 0.0585	15.09
GConvLSTM	0.3281 ± 0.2837	0.3061 ± 0.2790	0.5859 ± 0.2960	0.3590 ± 0.3547	5.15	0.3025 ± 0.0656	0.2959 ± 0.0773	0.5789 ± 0.0826	0.2121 ± 0.0592	26.72
GMAN	0.2950 ± 0.3072	0.2933 ± 0.3065	0.5828 ± 0.3074	0.3510 ± 0.3640	2.15	0.0772 ± 0.0203	**0.0838 ± 0.0240**	1.0951 ± 0.1900	0.0841 ± 0.0267	7.18
GraphWaveNet	0.3541 ± 0.2760	0.3949 ± 0.2359	0.7576 ± 0.2598	0.3860 ± 0.3282	1.30	0.8576 ± 0.0286	0.7169 ± 0.0194	0.8043 ± 0.0674	0.8152 ± 0.0219	4.34
LSGCN	**0.2945 ± 0.3047**	**0.2841 ± 0.3145**	**0.4103 ± 0.3481**	0.3377 ± 0.3727	1.98	0.1716 ± 0.0514	0.1717 ± 0.0523	**0.2315 ± 0.0769**	0.1180 ± 0.0300	12.61
MPNNLSTM	0.5047 ± 0.1753	0.4736 ± 0.1727	0.7037 ± 0.2140	0.4473 ± 0.2907	1.15	0.2558 ± 0.0316	0.4481 ± 0.0159	0.6909 ± 0.2219	0.2128 ± 0.0318	10.86
MTGNN	0.3072 ± 0.3025	0.3068 ± 0.2654	0.4384 ± 0.3544	0.3526 ± 0.3408	1.64	**0.0721 ± 0.0158**	0.1019 ± 0.0193	2.8671 ± 0.6181	**0.0369 ± 0.0059**	8.49
SLCNN	0.3218 ± 0.2862	0.3310 ± 0.2972	0.4537 ± 0.3133	0.3737 ± 0.3492	2.77	0.2806 ± 0.0628	0.3153 ± 0.0808	0.3392 ± 0.1077	0.2121 ± 0.0496	10.62
STAEformer	0.3273 ± 0.2879	0.3029 ± 0.2749	0.5769 ± 0.4392	0.3713 ± 0.3350	1.98	0.2148 ± 0.0495	0.2626 ± 0.0479	0.5042 ± 0.0850	0.2028 ± 0.0407	**3.21**
STGCN	0.3156 ± 0.2928	0.3043 ± 0.3012	0.4646 ± 0.3263	0.3568 ± 0.3588	5.54	0.3005 ± 0.0733	0.3167 ± 0.1014	0.6177 ± 0.4035	0.2077 ± 0.0566	28.39
STGNN	0.3079 ± 0.2947	0.2931 ± 0.2871	0.4364 ± 0.3245	0.3582 ± 0.3543	1.57	0.2680 ± 0.0650	0.2809 ± 0.0784	0.3233 ± 0.0899	0.1927 ± 0.0543	3.32
STGraformer	0.3043 ± 0.3104	0.2891 ± 0.3021	0.5263 ± 0.3190	**0.3376 ± 0.3704**	3.28	0.1010 ± 0.0292	0.1526 ± 0.0886	0.6179 ± 0.1229	0.0802 ± 0.0284	9.90
TGAT	0.3325 ± 0.2809	0.3838 ± 0.2895	0.6087 ± 0.3999	0.4504 ± 0.3262	**1.03**	0.2035 ± 0.0418	0.3877 ± 0.0143	0.3353 ± 0.1086	0.2641 ± 0.0381	3.45
TGCN	0.3311 ± 0.2810	0.3258 ± 0.2658	0.5757 ± 0.3015	0.3622 ± 0.3523	1.45	0.2928 ± 0.0608	0.3034 ± 0.0668	0.5198 ± 0.1820	0.2053 ± 0.0541	11.89

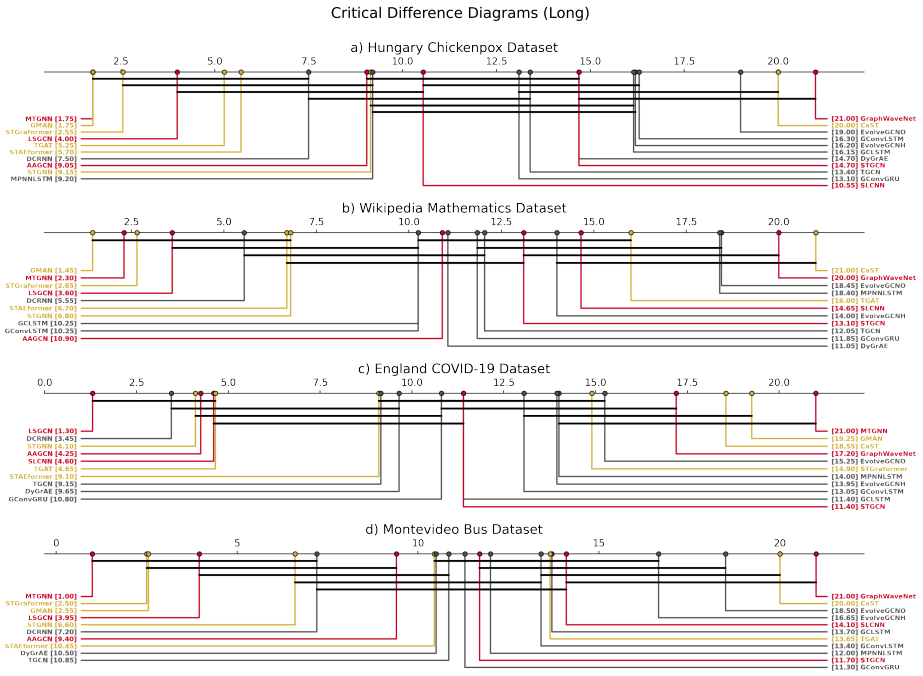


Fig. 2. Critical Difference (CD) diagrams for Long-Range forecasting regimes. The plots follow the same scheme as in Figure 1.

variability. Although this pattern is consistent with a spectral interpretation, the limited number of datasets prevents stronger conclusions.

In the Long-Range regime, performance differences become more pronounced, as shown in Figure 2 and Table 3. Architectures such as STGraformer and GMAN achieve lower RMSE values on Chickenpox and Montevideo Bus, while MTGNN and LSGCN also outperform several alternatives. These results suggest that extended forecasting horizons amplify architectural differences.

However, this advantage is not uniform. On EnglandCOVID, models such as GMAN and MTGNN exhibit degraded performance and high variance despite stronger results on other datasets. This indicates that spectral properties alone are insufficient to explain performance, as non-stationarity and temporal irregularities also play important roles.

Overall, Long-Range forecasting increases performance separation among architectures, but no single family consistently dominates across all settings. The observed spectral patterns help contextualize these differences, although they remain descriptive and require further investigation.

Finally, we analyze the impact of the input projection defined in Eq. (11). The ablation suggests that this projection generally improves performance, although its effect varies across architectures and datasets. Removing it worsens perfor-

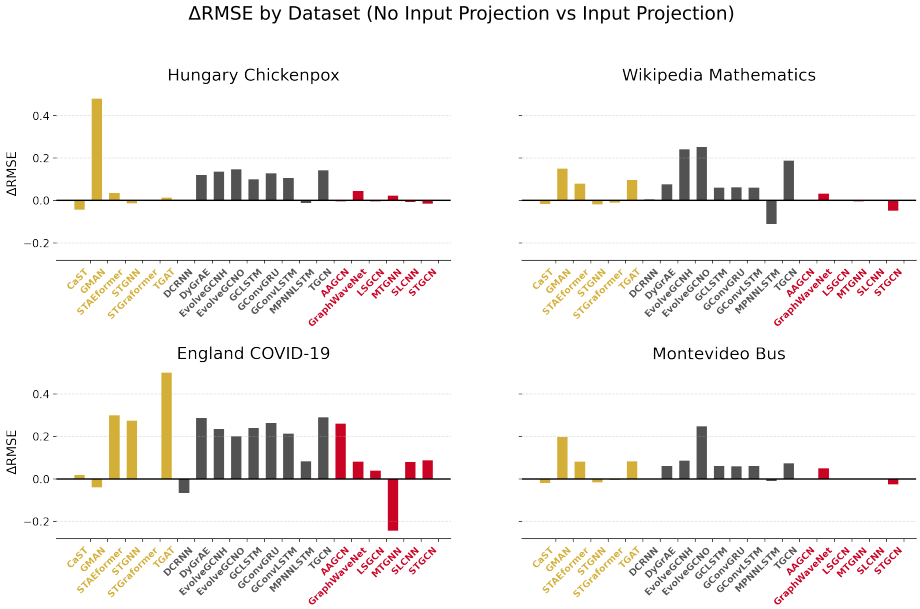


Fig. 3. Impact of removing the input projection in the Long-Range regime. Bars report the mean paired ΔRMSE over 10 seeds, computed as $\mathrm{RMSE}_{\mathrm{noIP}} - \mathrm{RMSE}_{\mathrm{IP}}$ for the same architecture, dataset, and seed. Positive values indicate worse performance without projection. Architectures are grouped by family: attention-based (yellow), recurrent (gray), and convolutional (red).

mance in about 63% of cases, as shown in Figure 3, with stronger degradation in recurrent models (86%) than in attention-based (54%) and convolutional models (38%). This suggests that recurrent models may be more sensitive to poorly conditioned inputs due to sequential hidden-state updates, while attention-based models may benefit from a projected latent space for similarity estimation and attention weighting. Convolutional models appear more robust due to local temporal filtering; however, this remains a descriptive trend, since dataset properties, graph topology, signal smoothness, internal projections, and optimization stability may also affect input conditioning.

4 Conclusions

This work evaluated 21 spatio-temporal graph neural network architectures under a unified pipeline across heterogeneous graph datasets. Results show no universally superior architecture; performance is strongly dataset-dependent. Convolutional models remain competitive, while attention-based models tend to help in long-range regimes with non-local dependencies. These findings partially support an interpretative spectral hypothesis: temporal mechanisms appear associ-

ated with spectral and structural properties of graph signals. However, broader validation on additional datasets is still needed.

Future work includes expanding the benchmark to datasets with more diverse structural and spectral profiles, refining spectral analysis with time-varying descriptors, directed operators, and horizon-specific errors, and testing alternative input representations, including scalar node inputs instead of lag vectors. Further studies may also explore ablations on input projection, sparsification, adaptive adjacency, receptive field, and spatial depth, as well as dataset-aware STGNNs that adapt to spectral and temporal characteristics.

Acknowledgements

The authors used ChatGPT for language revision and text editing. We thank Universidade Federal de Ouro Preto (UFOP), through PROPPI/UFOP, for supporting conference participation. This study was financed by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001.

References

1. Aileen Nielsen. *Practical Time Series Analysis*. O'Reilly Media, 2019.
2. Fernando H. O. Duarte et al. Leveraging graph neural networks and mobility data for COVID-19 forecasting. *Applied Soft Computing*, 198:115242, 2026.
3. Albert-László Barabási and Márton Pósfai. *Network Science*. Cambridge University Press, 2016.
4. Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
5. Lingfei Wu et al. *Graph Neural Networks: Foundations, Frontiers, and Applications*. Springer, 2022.
6. Yaguang Li et al. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations*, 2018.
7. Bing Yu et al. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, pages 3634–3640, 2018.
8. Zhijian Wu et al. Graph wavenet for deep spatial-temporal graph modeling. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pages 1907–1913, 2019.
9. Changan Zheng et al. GMAN: A graph multi-attention network for traffic prediction. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(01):1234–1241, 2020.
10. David I. Shuman et al. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. *IEEE Signal Processing Magazine*, 30(3):83–98, 2013.
11. Mark Newman. *Networks*. Oxford University Press, 2 edition, 2018.
12. Dragoš Cvetković et al. *Spectra of Graphs: Theory and Applications*. Academic Press, 1995.

13. Benjamin Ricaud et al. Fourier could be a data scientist: From graph fourier transform to signal processing on graphs. *Comptes Rendus Physique*, 20(5):474–488, 2019.
14. Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 2021.
15. Zonghan Wu et al. Connecting the dots: Multivariate time series forecasting with graph neural networks, 2020.
16. Rongzhou Huang et al. LSGCN: Long short-term traffic prediction with graph convolutional networks. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence*, pages 2355–2361, 2020.
17. Qiang Zhang et al. Spatio-temporal graph structure learning for traffic forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 1177–1185, 2020.
18. Bin Wang et al. AAGCN: A graph convolutional neural network with adaptive feature and topology learning. *Scientific Reports*, 14(1):10134, 2024.
19. Ling Zhao et al. T-GCN: A temporal graph convolutional network for traffic prediction. *IEEE Transactions on Intelligent Transportation Systems*, 21(9):3848–3858, 2020.
20. Youngjoo Seo et al. Structured sequence modeling with graph convolutional recurrent networks, 2016.
21. Jinyin Chen et al. GC-LSTM: Graph convolution embedded lstm for dynamic link prediction, 2021.
22. George Panagopoulos et al. Transfer graph neural networks for pandemic forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
23. Aynaz Taheri and Tanya Berger-Wolf. Predictive temporal embedding of dynamic graphs. In *Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*, pages 57–64, 2020.
24. Aldo Pareja et al. EvolveGCN: Evolving graph convolutional networks for dynamic graphs, 2019.
25. Yilong Jia et al. Temporal graph attention network for building thermal load prediction. *Energy and Buildings*, 321:113507, 2024.
26. Hangchen Liu et al. STAEformer: Spatio-temporal adaptive embedding makes vanilla transformer SOTA for traffic forecasting, 2023.
27. Xiaoyang Wang et al. Traffic flow prediction via spatial temporal graph neural network. In *Proceedings of The Web Conference 2020*, pages 1082–1092, 2020.
28. Chengxuan Ying et al. Do transformers really perform bad for graph representation?, 2021.
29. Yutong Xia et al. Deciphering spatio-temporal graph forecasting: A causal lens and treatment, 2023.
30. Benedek Rozemberczki et al. PyTorch Geometric Temporal: Spatiotemporal signal processing with neural machine learning models. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*, pages 4564–4573, 2021.
31. Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.
32. Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts, 2017.
33. Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937.
34. Peter Nemenyi. *Distribution-Free Multiple Comparisons*. PhD thesis, Princeton University, 1963.